

HW3 Theory & Documentation Reference

Part 1: Prompt Engineering Techniques

1. Zero-Shot Prompting

What it is: The model receives only a task instruction and the input — no worked examples. It relies entirely on knowledge encoded during pretraining/instruction-tuning.

Why it can fail: With nothing to anchor the format or reasoning depth expected, the model may jump straight to a final answer without intermediate computation — especially if the prompt explicitly asks for a short answer (as your `zero_shot_prompt_1` did: "give only the final numeric answer"). This suppresses the token-by-token reasoning that transformer LMs use to "think," which is the likely reason your zero-shot run produced 45 mph instead of the correct 42.86 mph — the model had no room to show/perform the intermediate division.

Key reference: Brown et al., *"Language Models are Few-Shot Learners"* (GPT-3 paper), 2020 — establishes the zero-shot / few-shot / one-shot terminology still used today.

2. Few-Shot Prompting

What it is: 2–3 worked input→output examples are prepended to the real question, in the same format, before the model sees the actual query. The model performs in-context learning: pattern-matching the demonstrated format/reasoning style without any weight updates.

Why it works: Transformer self-attention lets every token in the prompt (including the worked examples) influence the representation used to generate the next token. The examples effectively bias the model toward a particular output structure and level of shown work.

Key reference: Brown et al. (2020), same paper as above — few-shot ("in-context learning") is the paper's central contribution.

3. Chain-of-Thought (few-shot CoT)

What it is: Like few-shot, but the worked examples explicitly include step-by-step reasoning text before the final answer, rather than jumping straight to the answer.

Why it works: By demonstrating intermediate reasoning tokens, CoT prompting induces the model to generate its own intermediate steps for the new question — and because each generated token conditions on all previous tokens (autoregressive generation), correct intermediate steps make the final answer token more likely to be correct too. This is empirically shown to improve performance sharply on arithmetic, symbolic, and commonsense reasoning tasks, especially in larger models (an "emergent ability").

Key reference: Wei et al., *"Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,"* NeurIPS 2022.

4. Zero-Shot Chain-of-Thought

What it is: No worked examples at all. Instead, a fixed trigger phrase — "Let's think step by step" — is appended to an otherwise bare zero-shot prompt.

Why it works: The trigger phrase alone is enough to shift the model's generation distribution toward producing an explicit reasoning trace before the final answer, recovering much of CoT's benefit without needing hand-written

examples. This suggests the capability for step-by-step reasoning is already present in the pretrained model and mainly needs to be elicited, not taught.

Key reference: Kojima et al., "*Large Language Models are Zero-Shot Reasoners*," NeurIPS 2022.

5. Meta-Prompting

What it is: Rather than asking the model to solve an instance directly, you first ask it to articulate a general procedure/strategy for a whole class of problems, then in a second call apply that procedure to the specific instance.

Why it can still fail: A syntactically sound procedure does not guarantee correct execution of that procedure. Your Meta-Prompting run is a good illustration: step 1 produced a logically correct general method for evaluating syllogisms, but step 2 misapplied it, declaring a valid syllogism ("Some employees are managers; all managers attended the retreat; therefore some employees attended") invalid. This is exactly the classical valid syllogism form Darii (All M are P, Some S are M $\vdash$ Some S are P) — the model had the right recipe but made a reasoning error executing it. This is worth citing directly in your [Discussion](#) as a genuine, task-specific failure mode of meta-prompting: having a correct high-level plan doesn't guarantee correct step-by-step application.

Key reference: the term "meta-prompting" is used more informally across recent LLM prompting literature (e.g., Suzgun & Kalai, "*Meta-Prompting for AI Systems*," 2024) to describe this plan-then-execute or self-generated-instruction pattern, distinct from the more formally studied CoT/ToT techniques above.

6. Tree of Thoughts (ToT)

What it is: Generalizes CoT from a single linear reasoning chain to a search over a tree of intermediate "thoughts," where multiple candidate next-steps are generated, evaluated, and pruned — allowing backtracking, unlike CoT's one-shot linear path. The full algorithm uses BFS/DFS-style search across a language model's own self-evaluations of each branch.

Why your single-call simulation is a reasonable approximation: True ToT requires the outer program (not the LLM alone) to do the search/pruning/backtracking across multiple separate LLM calls. Asking the model to enumerate branches, reason through each, self-evaluate, and pick a winner within one call captures the "explore multiple candidate solutions and evaluate them" spirit, but without genuine backtracking or search-tree pruning across calls. This is worth stating explicitly as a limitation in your write-up.

Note on your actual results: the math-problem branches were not independent explorations — all three used the same total-distance/total-time computation [restated](#) differently, so no real divergent search happened. The logic-puzzle branches, in contrast, were genuinely distinct candidate solutions (different friend→pet assignments), each checked and pruned against the stated constraints with a specific, non-redundant justification — a much closer approximation to real ToT behavior. This asymmetry (constraint-satisfaction puzzles inducing genuine branching vs. straightforward arithmetic not needing it) is itself a useful, citable observation.

Key reference: Yao et al., "*Tree of Thoughts: Deliberate Problem Solving with Large Language Models*," NeurIPS 2023.

Part 2: Self-Attention and Causal Masking

Scaled Dot-Product Attention

Definition (Vaswani et al., 2017, Section 3.2):

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$$

- **Q (query), K (key), V (value)** are all linear projections of the same input sequence in self-attention (as opposed to cross-attention, where Q comes from one sequence and K/V from another).

QK^T computes a similarity score between every pair of positions — this is why attention is fundamentally content-based: it depends on what the tokens are (via their learned embeddings), not directly on their position, except insofar as position is baked into the embedding via a positional encoding/embedding added beforehand.

- **Division by $\sqrt{d_k}$** (the scaling) prevents the dot products from growing too large in magnitude as dimensionality increases, which would otherwise push softmax into regions with vanishingly small gradients.
- **softmax** turns raw scores into a probability distribution over positions (each row sums to 1) — the "attention weights" you visualized as a heatmap.
- The output at each position is a weighted average of all Value vectors, weighted by how much that position attends to each other position.

Why your unmasked heatmap showed content-matching rather than "peek at position $i+1$ ": since attention operates on $Q \cdot K$ similarity of token content embeddings, and your text has repeated words, the easiest signal for the (randomly initialized, then gradient-trained) Q/K projections to discover is "attend to other occurrences of the same word," which produces high dot-product similarity naturally — this is a simpler pattern for optimization to find than a precise "+1 positional offset" relationship, which depends on the added positional embedding component being learned in a very specific, position-dependent way.

Causal (Masked) Self-Attention

Definition: Before the softmax, all entries of the score matrix above the diagonal (i.e., position i attending to any position $j > i$) are set to $-\infty$, so that after softmax they receive exactly zero probability. This restricts each position to attending only to itself and earlier positions — required for autoregressive generation, since at inference time the model genuinely cannot see future tokens (they haven't been generated yet), so training must reflect that same constraint to avoid a train/inference mismatch (the model would otherwise learn to rely on information it won't have access to at generation time).

Reference: Vaswani et al., "Attention Is All You Need," NeurIPS 2017 — decoder self-attention sublayer description, Section 3.2.3: "We need to prevent leftward information flow in the decoder to preserve the auto-regressive property."

Quick Citation List

- Vaswani, A., et al. (2017). *Attention Is All You Need*. NeurIPS.
- Brown, T., et al. (2020). *Language Models are Few-Shot Learners*. NeurIPS.
- Wei, J., et al. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. NeurIPS.
- Kojima, T., et al. (2022). *Large Language Models are Zero-Shot Reasoners*. NeurIPS.
- Yao, S., et al. (2023). *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*. NeurIPS.
- Suzgun, M., & Kalai, A. (2024). *Meta-Prompting for AI Systems*. arXiv.